

- Xebia — Principal Architect
- L2 Final Round Interview Preparation
 - What Xebia Final Rounds Actually Test
 - Table of Contents
 - 1. Live Architecture Design Sessions
 - Design Session 1: "A fintech client wants to migrate from a monolithic Java app to microservices on AWS. They have 50 developers, 200K customers, \$5M annual cloud budget. Design the target architecture."
 - Design Session 2: "Design a multi-tenant SaaS platform for an enterprise client that needs to serve customers in India, US, and EU with data residency requirements."
 - Design Session 3: "A healthcare client needs a real-time patient monitoring system. 10,000 devices sending vitals every second. Alerts must fire within 3 seconds of an anomaly."
 - 2. Deep-Dive Technical Probes
 - Q1: "You mentioned Istio service mesh in L1. Walk me through exactly how mTLS works between two pods, step by step."
 - Q2: "You use Karpenter. What happens internally when a pod is pending and no node has capacity? Walk me through Karpenter's decision process."
 - Q3: "Explain the CAP theorem with a practical example. When would you choose AP over CP?"
 - 3. Consulting & Client Engagement Scenarios
 - Q4: "A client's CTO insists on using Azure because their CEO plays golf with the Microsoft sales team. Your assessment says AWS is better for their workload. What do you do?"
 - Q5: "You're 3 months into a 6-month engagement. The client's architects are resistant to your recommendations. They see you as an outsider. How do you turn this around?"
 - Q6: "A client wants you to scope a cloud transformation engagement. How do you approach the initial discovery and proposal?"
 - Q7: "A client's VP of Engineering asks you: 'We're spending \$2M/year on AWS and our CTO thinks it's too much. Can you help us cut 30%?' How do you approach this?"
 - 4. Debugging & Incident Scenarios
 - Q8: "A microservice is returning intermittent 504 Gateway Timeout errors. It works fine 95% of the time. How do you diagnose it?"

- 5. Trade-off & Decision-Making
 - Q9: "Kafka vs. SQS+SNS vs. EventBridge — how do you choose for a specific client?"
 - Q10: "Monorepo vs. polyrepo for 50+ microservices — what's your recommendation?"
- 6. Architecture Leadership at Scale
 - Q11: "How do you introduce a new technology (e.g., service mesh) into an organization that's resistant to change?"
 - Q12: "How do you measure the health of an engineering organization's architecture?"
- 7. Xebia Culture & Values Fit
 - Q13: "What does 'craftsmanship' mean to you in the context of software architecture?"
 - Q14: "How do you contribute to the broader engineering community? (Open source, talks, writing, mentoring)"
- 8. Behavioral / STAR Stories
 - Story 1: "Tell me about a time you inherited a failing project and turned it around."
 - Story 2: "Tell me about a time you had to say 'no' to a client."
- 9. VP/CTO Rapid-Fire Questions
- 10. Questions YOU Should Ask
- Final Interview Prep Checklist
- The Difference Between L1 and L2 at Xebia

Xebia — Principal Architect

L2 Final Round Interview Preparation

Round: Final F2F | **Interviewer:** VP Engineering / Practice Head / CTO **Format:** 60–90 min | Architecture deep-dive + Live design + Consulting scenarios + Cultural fit **Builds on:** [Xebia_Principal_Architect_Interview_Prep.md](#) (L1 — 70 Q&As) **What L2 tests:** How you THINK, how you APPROACH client problems, how you LEAD across organizations

What Xebia Final Rounds Actually Test

Xebia is a **consulting/services company** — not a product company. The final round assesses whether you can:

Dimension	What They Probe
Client-facing architecture	Can you whiteboard for a CTO who doesn't know Kubernetes?
Consulting maturity	Can you handle ambiguity, scope creep, difficult clients?
Solution approach	Do you start with "why" or jump to "how"?
Technical depth + breadth	Can you go deep on any topic the interviewer picks?
Xebia culture fit	Craftsmanship, continuous learning, knowledge sharing
Revenue-thinking	Can you see the next engagement inside the current one?
Architecture at scale	50+ team, multi-cloud, multi-region thinking
Influence without authority	Client teams don't report to you — can you still drive change?

Table of Contents

1. [Live Architecture Design Sessions](#)
2. [Deep-Dive Technical Probes](#)
3. [Consulting & Client Engagement Scenarios](#)
4. [Debugging & Incident Scenarios](#)
5. [Trade-off & Decision-Making](#)
6. [Architecture Leadership at Scale](#)
7. [Xebia Culture & Values Fit](#)
8. [Behavioral / STAR Stories](#)
9. [VP/CTO Rapid-Fire Questions](#)
10. [Questions YOU Should Ask](#)

1. Live Architecture Design Sessions

The interviewer will give you an open-ended problem. They're watching **your process**, not your final answer.

Design Session 1: "A fintech client wants to migrate from a monolithic Java app to microservices on AWS. They have 50 developers, 200K customers, \$5M annual cloud budget. Design the target architecture."

Your framework (say this out loud):

"Before I design anything, let me understand the constraints and goals."

DISCOVERY QUESTIONS I'D ASK:

- What's the business driver? (Scale? Speed-to-market? Compliance? Cost?)
- What's their current pain? (Deployment frequency? Outage frequency? Lead time?)
- Regulatory: PCI-DSS? RBI? SOC2? (Fintech = heavy compliance)
- Current state: What Java framework? (Spring Boot? J2EE? Struts?)
- Database: Oracle? PostgreSQL? How coupled is business logic to DB?
- Team structure: 50 devs – how many teams? Cross-functional or siloed?
- Timeline: 6 months? 18 months? Is there a hard deadline?
- What have they already tried? (Failed migrations often have political baggage)

Phase 1: Assess & Plan (Week 1–4)

Domain Analysis (Event Storming):

- Run 3-day Event Storming workshop with domain experts
- Identify bounded contexts: Payments, KYC, Lending, Accounts, Notifications
- Map: which contexts are tightly coupled today?
- Identify: which context changes most often? (extract first)
- Output: Domain map + service boundary proposal

Technical Assessment:

- Code analysis: identify hot spots (most-changed files = extraction candidates)
- Database analysis: table dependencies, stored procedures, triggers

- └ Performance baseline: current p95 latency, throughput, error rate
- └ Dependency mapping: what talks to what? (Jaeger/ServiceMesh or manual)
- └ Output: Migration complexity matrix (effort vs. risk per component)

Phase 2: Foundation (Week 5–12)

AWS Landing Zone:

- └ AWS Organizations: prod, staging, security, shared-services accounts
- └ Networking: VPC with private subnets, Transit Gateway, VPC endpoints
- └ EKS: Private cluster, Karpenter, Graviton nodes
- └ Security: IAM Identity Center, KMS CMK, GuardDuty, Security Hub
- └ CI/CD: GitHub Actions → ECR → ArgoCD (GitOps)
- └ Observability: OpenTelemetry → Grafana LGTM stack

PCI-DSS Compliance Architecture:

- └ Cardholder Data Environment (CDE) in isolated VPC
- └ Tokenization: card numbers never stored – use Stripe/payment gateway
- └ Network segmentation: CDE has no direct internet access
- └ Audit logging: CloudTrail → immutable S3 (WORM via Object Lock)
- └ WAF: AWS WAF with OWASP ruleset on ALB
- └ Quarterly PCI scans: automated via Inspector + third-party ASV

Phase 3: Strangler Fig Migration (Week 12–52)

Migration sequence (highest-value, lowest-risk first):

Wave 1 (Month 3–5): Notifications Service

- └ WHY first: least coupled, lowest risk, high change frequency
- └ Extract from monolith → new Spring Boot microservice on EKS
- └ API Gateway routes /api/v2/notifications → new service
- └ Monolith still handles everything else
- └ Validate: same functionality, better deploy speed (daily vs monthly)
- └ Team learns the new stack on a safe service

Wave 2 (Month 5–8): KYC / Onboarding

- └ High business value (new customer acquisition)
- └ Document verification → AWS Textract + custom ML
- └ Event-driven: KYC completed → publish event → other services react
- └ Separate database (PostgreSQL on Aurora)

Wave 3 (Month 8–12): Payments

- └ Highest risk – PCI scope; extract carefully
- └ Saga pattern: Order → Reserve → Charge → Fulfill (Step Functions)
- └ Database migration: Oracle stored procs → application logic + Aurora
- └ Shadow testing: run old + new in parallel, compare results
- └ Canary deployment: 1% → 5% → 25% → 100% over 2 weeks

Wave 4–N (Month 12+): Remaining services

- └ Accounts, Lending, Reporting

- └ By now team is experienced with the pattern
- └ Monolith shrinks progressively; eventually decommissioned

What makes this answer Principal-level:

- You didn't jump to the architecture — you asked discovery questions first
- You sequenced by risk and value, not by technical elegance
- You addressed PCI compliance proactively (fintech = regulated)
- You proposed Strangler Fig (incremental) not big-bang (risky)
- You mentioned team learning curve as a factor in sequencing

Design Session 2: "Design a multi-tenant SaaS platform for an enterprise client that needs to serve customers in India, US, and EU with data residency requirements."

Architecture:

MULTI-REGION MULTI-TENANT ARCHITECTURE:

GLOBAL LAYER (Region-agnostic):

- └ Route 53: Geolocation routing
 - └ *.in.platform.com → ap-south-1 (Mumbai)
 - └ *.us.platform.com → us-east-1 (Virginia)
 - └ *.eu.platform.com → eu-west-1 (Ireland)
- └ CloudFront: Global CDN (static assets, API caching)
- └ Global API Gateway: tenant onboarding, routing metadata
- └ Control Plane: tenant registry, configuration, billing (single region)

PER-REGION DATA PLANE:

- ap-south-1 (India)
 - └ EKS Cluster (private, multi-AZ)
 - └ Shared Namespace: Pool tenants (SMB)
 - └ RLS in Aurora: tenant_id per row
 - └ Dedicated Namespace: Enterprise tenants
 - └ Separate Aurora cluster per tenant
 - └ Aurora PostgreSQL (Multi-AZ, encrypted KMS)
 - └ ElastiCache Redis (session, cache)
 - └ S3 Data Lake (Bronze/Silver/Gold per tenant)
 - └ Kinesis (event streaming – stays in region)
 - └ ALL DATA STAYS IN THIS REGION (RBI compliance)

```
eu-west-1 (Ireland)
├─ Same architecture as above
└─ ALL DATA STAYS IN THIS REGION (GDPR)
```

```
us-east-1 (Virginia)
├─ Same architecture as above
├─ Control Plane (master): tenant registry, billing
│   └─ Replicated metadata to other regions (no PII)
└─ Cross-region: only anonymized metrics replicated
```

DATA RESIDENCY ENFORCEMENT:

```
├─ SCP: Deny s3:PutObject to buckets outside assigned region
├─ SCP: Deny ec2:RunInstances outside assigned region
├─ Terraform: region variable enforced per tenant account
├─ Application: tenant_region in JWT → reject API calls to wrong region
└─ Audit: AWS Config rule → alert if data resource created in wrong region
```

CROSS-REGION CHALLENGES:

```
├─ Global user directory: Cognito user pool per region (no cross-region
federation)
│   └─ Solution: user created in their region; if they travel, API Gateway
routes
│       to home region based on tenant_id (not geo-IP)
├─ Global analytics: anonymized, aggregated metrics sent to central region
│   └─ No PII crosses borders; only counts and averages
├─ Disaster recovery: within-region multi-AZ (not cross-region DR for data)
│   └─ Reason: cross-region DR violates data residency; accept regional
RPO/RT0
└─ Deployment consistency: same ArgoCD manifests deployed to all regions
    └─ GitOps repo → ApplicationSet → targets all 3 clusters
```

Why this answer wins at Principal level:

- Data residency is treated as a first-class constraint, not an afterthought
- You proposed a tiered tenancy model (Pool + Dedicated) for economics
- You addressed the cross-region analytics challenge (anonymized aggregation)
- You identified that cross-region DR conflicts with data residency and made an explicit trade-off decision
- You showed SCP enforcement (preventive, not detective only)

Design Session 3: "A healthcare client needs a real-time patient monitoring system. 10,000 devices sending vitals every second. Alerts must fire within 3 seconds of an anomaly."

REAL-TIME PATIENT MONITORING ARCHITECTURE:

DEVICE LAYER:

- Medical devices ($10,000 \times 1$ reading/sec = 10,000 events/sec)
- Protocol: MQTT (lightweight, persistent connections)
- Edge gateway per hospital ward: AWS IoT Greengrass
 - Local anomaly detection (ML model at edge for <100ms alerting)
 - Buffer during network outage (store-and-forward)
 - Data filtering: only send abnormal + 1/10th normal readings to cloud
- TLS mutual authentication (device certificates via IoT Core)

INGESTION:

- AWS IoT Core: 10,000 MQTT connections, managed broker
 - IoT Rule 1 → Kinesis Data Streams (raw telemetry for processing)
 - IoT Rule 2 → Lambda (threshold breach → immediate SNS alert)
 - IoT Rule 3 → S3 (raw archive for compliance, 7-year retention)
- Kinesis Data Streams: 10 shards ($10,000$ events/sec $\times$ 200 bytes)
- Kinesis Data Analytics (Apache Flink):
 - Sliding window (5 seconds): detect sustained anomaly
 - Pattern detection: heart rate spike + blood pressure drop = CODE

BLUE

- Output: DynamoDB (real-time dashboard) + SNS (clinical alert)

ALERT PIPELINE (< 3 second SLA):

- Path 1 – EDGE (< 500ms):
 - Greengrass ML model detects anomaly → local alert to ward nurse station (works even if cloud connectivity is lost)
- Path 2 – CLOUD (< 3 seconds):
 - IoT Core → Kinesis → Flink (pattern detection) → SNS → clinical staff
 - Escalation: if not acknowledged in 60s → page on-call physician
- Alert delivery:
 - Push notification to clinical mobile app
 - Pager / SMS (backup for mobile)
 - Dashboard: real-time patient vitals on ward display

COMPLIANCE (HIPAA):

- PHI (Protected Health Information) encrypted at rest and in transit
- BAA (Business Associate Agreement) with AWS
- VPC: no internet access for data processing tier
- Audit trail: CloudTrail + IoT Core logs → immutable S3
- Access control: patient data = ABAC (attribute-based access control)
 - Doctor sees their patients only; nurse sees their ward only
- Data retention: 7 years minimum (HIPAA requirement)

AVAILABILITY:

- 99.99% uptime SLA (life-critical system)
- Multi-AZ everything (no single point of failure)
- Edge fallback: if cloud is unreachable, edge continues monitoring + alerting
- No planned downtime: blue/green deployments, rolling updates
- Chaos testing: monthly failure injection with clinical team awareness

2. Deep-Dive Technical Probes

In L2, the interviewer picks one topic from L1 and goes very deep. Prepare to go 3-4 levels deeper than L1.

Q1: "You mentioned Istio service mesh in L1. Walk me through exactly how mTLS works between two pods, step by step."

Answer:

ISTIO mTLS – WHAT ACTUALLY HAPPENS:

1. SETUP (at deployment time):

- Istiod (control plane) has a CA (Certificate Authority)
- Each pod gets an Envoy sidecar injected (via admission webhook)
- Envoy requests a certificate from Istiod via SDS (Secret Discovery Service)
 - Istiod issues a short-lived X.509 cert (SPIFFE identity):
spiffe://cluster.local/ns/prod/sa/order-service
 - Cert rotated every 24 hours (no manual renewal)
 - Private key never leaves the pod (generated in Envoy)

2. REQUEST FLOW (Service A → Service B):

Pod A (order-service)

- App code calls `http://payment-service:8080/charge` (plain HTTP)
- Request intercepted by Envoy sidecar (iptables redirect)
- Envoy A initiates TLS handshake with Envoy B:
 - Envoy A sends ClientHello (TLS 1.3)
 - Envoy B responds with its certificate
Subject: `spiffe://cluster.local/ns/prod/sa/payment-service`
 - Envoy A verifies cert against Istiod's root CA
 - Envoy A sends its certificate (mutual = client also authenticates)

- Envoy B verifies Envoy A's cert against root CA
- TLS session established (encrypted, both identities verified)
- Request forwarded over encrypted channel to Envoy B
- Envoy B decrypts → forwards to `payment-service` app on `localhost:8080`
- App code is unaware of TLS (Envoy handles everything transparently)

3. AUTHORIZATION (after mTLS):

- Istio AuthorizationPolicy:
apiVersion: `security.istio.io/v1`
kind: `AuthorizationPolicy`
metadata:

```
name: payment-service-policy
namespace: prod
spec:
  selector:
    matchLabels:
      app: payment-service
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/prod/sa/order-service"]
          to:
            - operation:
                methods: ["POST"]
                paths: ["/charge"]
```

└─ Effect: Only order-service can POST /charge. All other services → 403.

4. OBSERVABILITY:

- └─ Envoy emits metrics: request count, latency, error rate (to Prometheus)
- └─ Distributed trace headers propagated through Envoy (to Jaeger/X-Ray)
- └─ Access logs: source identity, destination, status code, latency

Why this matters at Principal level:

"mTLS gives us zero-trust networking without application code changes. Every service proves its identity on every call. Combined with AuthorizationPolicy, we get microsegmentation at L7 — far more granular than NetworkPolicy (L3/L4). For Xebia's consulting clients in regulated industries, this is the difference between a 6-month compliance audit and a 2-week audit."

Q2: "You use Karpenter. What happens internally when a pod is pending and no node has capacity? Walk me through Karpenter's decision process."

Answer:

KARPENTER INTERNAL DECISION FLOW:

1. TRIGGER:

- └─ kube-scheduler can't find a node for a pending pod
- └─ Pod stays in Pending state with reason: Unschedulable

- └─ Karpenter controller watches for Unschedulable pods (informer cache)

2. BATCHING (Karpenter waits 1-10 seconds):

- └─ Collects all pending pods in the batch window
- └─ Groups pods by compatible requirements (arch, instance family, zone)
- └─ Reason: launching 1 node for 5 pods is cheaper than 5 nodes for 5 pods

3. REQUIREMENT EVALUATION:

For each pod group, Karpenter evaluates:

- └─ Pod resource requests (CPU, memory, GPU)
- └─ NodePool constraints (from your NodePool CRD):
 - └─ karpenter.sh/capacity-type: [on-demand, spot]
 - └─ kubernetes.io/arch: [amd64, arm64]
 - └─ karpenter.k8s.aws/instance-category: [m, c, r]
 - └─ karpenter.k8s.aws/instance-generation: [>5]
- └─ Pod topology spread constraints
- └─ Pod node affinity/anti-affinity
- └─ Pod tolerations
- └─ Taints on the NodePool

4. INSTANCE TYPE SELECTION:

- └─ Karpenter evaluates ALL matching instance types (can be 100+)
- └─ Scores each by: price, available capacity, resource fit
- └─ For Spot: selects diversified set (15+ instance types for best availability)
- └─ For On-Demand: selects cheapest that fits
- └─ Bin-packing: choose instance size that fits all grouped pods tightly
- └─ Example: 3 pods needing 1 CPU + 2GB each → m6g.large (2 CPU, 8GB) not m6g.xlarge

5. EC2 API CALL:

- └─ Karpenter calls EC2 CreateFleet API directly (not ASG!)
 - └─ This is why it's faster than Cluster Autoscaler (no ASG intermediary)
- └─ Specifies: subnet, security group, user data, instance types (diversified)
- └─ Instance launches with EKS-optimized AMI
- └─ Kubelet joins the cluster (bootstrap via user data)

6. NODE READY (30-60 seconds total):

- └─ EC2 instance launches (~20-30 seconds)
- └─ Kubelet registers with API server (~10 seconds)
- └─ Node transitions to Ready
- └─ kube-scheduler schedules the pending pods onto the new node
- └─ Pods start running

7. CONSOLIDATION (ongoing):

- └─ Karpenter continuously evaluates: "Can I reduce the number of nodes?"
- └─ If a node is <50% utilized AND pods can fit on other nodes:
 - └─ Karpenter cordons the node
 - └─ Drains pods (respecting PDBs)
 - └─ Pods reschedule to other nodes
 - └─ Empty node terminated
- └─ This is how Karpenter saves 30-40% vs. Cluster Autoscaler

Q3: "Explain the CAP theorem with a practical example. When would you choose AP over CP?"

Answer:

CAP THEOREM:

In a distributed system experiencing a network partition (P), you must choose between:

Consistency (C): Every read gets the most recent write

Availability (A): Every request gets a response (even if stale)

You ALWAYS get partition tolerance in distributed systems.

The real choice is: C or A during a partition.

PRACTICAL EXAMPLE – E-Commerce Platform:

CP Choice: Payment Service

- └─ During a network partition between payment nodes:
 - └─ REFUSE to process payments rather than risk double-charging
 - └─ Return 503: "Payment temporarily unavailable"
 - └─ Customer sees error; retry later
- └─ Why CP: financial integrity > availability
- └─ Technology: Aurora PostgreSQL (synchronous replication within AZ)
- └─ Trade-off: brief unavailability during failover (~30s)

AP Choice: Product Catalog / Recommendations

- └─ During a network partition:
 - └─ CONTINUE serving product pages (maybe with stale prices)
 - └─ Show cached product data (might be 5 minutes old)
 - └─ Customer sees products; order might fail at checkout (handled gracefully)
- └─ Why AP: showing something > showing nothing
- └─ Technology: DynamoDB Global Tables (eventually consistent reads) or ElastiCache Redis (read replicas)
- └─ Trade-off: stale data for seconds-to-minutes

AP Choice: Shopping Cart

- └─ During a partition:
 - └─ Accept cart additions locally
 - └─ Merge conflicts later (last-write-wins or union-merge)
 - └─ Customer never loses items; might see duplicates briefly
- └─ Technology: DynamoDB (eventual consistency)
- └─ Trade-off: temporary inconsistency in cart contents

REAL-WORLD NUANCE:

"In practice, I design systems with tunable consistency per operation:

- Write path: strongly consistent (CP-ish)
- Read path: eventually consistent (AP-ish) with cache
- Critical reads (balance check before payment): strongly consistent
- Non-critical reads (product listing): eventually consistent

DynamoDB offers both modes per query. Aurora offers read replicas with replication lag < 50ms."

3. Consulting & Client Engagement Scenarios

Xebia is a consulting company. These scenarios test your consulting DNA.

Q4: "A client's CTO insists on using Azure because their CEO plays golf with the Microsoft sales team. Your assessment says AWS is better for their workload. What do you do?"

Answer:

"This is a common consulting scenario. The worst thing I can do is fight the decision head-on. Here's my approach:

Step 1: Understand the real constraint. The golf friendship isn't the reason — there's usually a deeper factor: existing Microsoft EA (Enterprise Agreement), Entra ID/Office 365 investment, or a genuine relationship with the Azure account team that provides support and discounts.

Step 2: Validate my own assessment. Am I sure AWS is better? What are the criteria?

Assessment Matrix:			
Criteria	AWS	Azure	Winner
-----	-----	-----	-----
Kubernetes (EKS vs AKS)	EKS	AKS (free CP)	Tie
ML/AI services	SageMaker	Azure ML	AWS (broader)
Existing team skills	AWS	None	AWS
Enterprise Agreement	None	Active EA	Azure

SS0/Identity	Separate	Entra ID	Azure	
Total cost (with EA)	\$X	\$X-30%	Azure	

Step 3: Present data, not opinions. 'Based on our assessment, AWS has stronger ML services for your use case. However, Azure closes this gap with Azure ML, and your existing Enterprise Agreement saves 30% on compute. Here's a comparison matrix. Both are viable.'

Step 4: Propose a path that works for everyone. 'I recommend we start with Azure for core infrastructure (leveraging your EA discount and Entra ID), and evaluate AWS for specific ML workloads where Azure capabilities are limited. We use Terraform and Kubernetes for portability — if we need to add AWS later, the migration is low-effort.'

Step 5: Document the decision. ADR with trade-offs. If Azure causes problems later, we have a record of the alternatives considered.

Key principle: A consulting architect's job isn't to choose the technically 'best' solution. It's to choose the solution that delivers maximum business value within the client's constraints. Sometimes that means a technically second-best choice that the organization will actually adopt."

Q5: "You're 3 months into a 6-month engagement. The client's architects are resistant to your recommendations. They see you as an outsider. How do you turn this around?"

Answer:

"This is the most common consulting failure mode. It usually happens because the consultant presented solutions instead of earning trust first.

Week 1–2 Recovery:

1. **Stop recommending.** Start asking. 'Help me understand why you chose this approach.' Their current architecture exists for reasons — some good, some

historical, some political.

2. **Pair with their best architect.** Not to 'teach' — to learn. Work on a real ticket together. Shared foxholes build trust.
3. **Win a small victory together.** Pick a pain point they've complained about (slow CI pipeline, flaky tests, noisy alerts). Fix it together in a week. Nothing builds credibility like solving real pain.

Week 3–4 Pivot: 4. **Reframe my role.** Instead of 'I'm here to redesign your architecture,' say 'I'm here to help YOU build the architecture YOU want. My job is to bring patterns I've seen at other companies and help you evaluate whether they fit.' 5. **Attribute ideas to them.** In meetings: 'As Rajesh suggested, we could use event-driven patterns here — I've seen this work well at a similar scale.' Now Rajesh is the champion, not me. 6. **Create a joint Architecture Decision Record.** Co-authored. Both names on it. Shared ownership of the direction.

What I'd do differently from day 1 next time:

- Spend week 1 observing and asking, not presenting
- Shadow their on-call rotation to understand their operational reality
- Never use the word 'should' in the first month — use 'what if we explored...'
- Build a relationship before building an architecture

The consulting truth: You can have the best architecture in the world. If the client's team doesn't believe in it, it will fail after you leave."

Q6: "A client wants you to scope a cloud transformation engagement. How do you approach the initial discovery and proposal?"

Answer:

DISCOVERY FRAMEWORK (2–3 WEEK ENGAGEMENT):

Week 1: UNDERSTAND

— Day 1–2: Stakeholder interviews (CTO, VPs, lead architects, ops leads)

Questions:

- "What's the business outcome you need from this transformation?"
- "What have you already tried? What worked? What didn't?"
- "What does success look like in 6 months? 18 months?"

- "What's your biggest fear about this transformation?"
 - "What's the one thing that would make your teams' lives easier tomorrow?"
- Day 3–4: Technical assessment
 - Application portfolio: number, languages, dependencies, deployment method
 - Infrastructure inventory: on-prem, cloud (which), hybrid
 - Data landscape: databases, data flows, volume, compliance requirements
 - Security posture: current tools, compliance certifications, gaps
 - Team assessment: skills, capacity, culture (DevOps maturity model)
- Day 5: Developer experience assessment
 - How long from code commit to production? (days? weeks? months?)
 - How many manual steps in deployment?
 - How do developers get a new environment?
 - What percentage of time is spent on toil vs. feature work?

Week 2: ANALYZE

- Current state architecture diagram (as-is)
- Pain point heatmap (severity × frequency)
- Cloud readiness assessment per application (6 Rs)
- Risk register: technical, organizational, compliance
- Quick wins list (deliverable in < 4 weeks, visible impact)
- Dependency map: what must move together

Week 3: PROPOSE

- Target state architecture (to-be, 18-month vision)
- Migration roadmap (phased waves with dependencies)
- Team structure recommendation (platform team, stream-aligned teams)
- Toolchain recommendation (IaC, CI/CD, observability, security)
- Investment model:
 - Phase 1 (0–6 months): Foundation – \$X, team of Y
 - Phase 2 (6–12 months): Migration – \$X, team of Y
 - Phase 3 (12–18 months): Optimization – \$X, team of Y
- ROI projection: current cost vs. target cost + productivity gains
- Risk mitigation plan
- Success metrics (DORA metrics baseline → target)

Xebia-specific approach:

"The discovery itself should be a value-delivery engagement, not just a report. I ensure the client gets at least 2–3 quick wins implemented during discovery — maybe a Terraform baseline, a CI pipeline improvement, or a security vulnerability fix. This demonstrates competence and builds trust for the larger engagement."

Q7: "A client's VP of Engineering asks you: 'We're spending \$2M/year on AWS and our

CTO thinks it's too much. Can you help us cut 30%?' How do you approach this?"

Answer:

FINOPS ENGAGEMENT APPROACH:

WEEK 1: VISIBILITY (you can't cut what you can't see)

- Enable Cost Explorer with hourly granularity
- Tag compliance audit: how many resources are untagged?
(untagged = unaccountable = waste)
- Set up Kubecost on EKS clusters (per-namespace, per-team cost)
- Install Infracost in CI (cost estimate on every Terraform PR)
- Quick analysis: top 10 cost drivers (usually 80% of bill)

TYPICAL FINDINGS (in my experience):

- 15–25%: Right-sizing (instances too large for actual usage)
 - Compute Optimizer recommendations
 - Dev/staging instances same size as prod (should be half)
- 10–15%: Unused resources
 - Idle EC2 instances (< 5% CPU for 7+ days)
 - Unattached EBS volumes (\$0.10/GB/month × forgotten)
 - Over-provisioned RDS (Multi-AZ on dev environments)
 - NAT Gateway data charges (use VPC endpoints instead)
- 10–20%: No commitment discounts
 - No Savings Plans or Reserved Instances
 - 3-year Compute SP saves up to 60%
 - 1-year RDS RI saves 40%
- 5–15%: Architecture inefficiency
 - No caching (every request hits DB instead of Redis)
 - Synchronous processing where async would work
 - Data transferred across AZs unnecessarily
- 5–10%: Non-prod waste
 - Dev/staging running 24/7 (only used 8/5)
 - Automated shutdown saves 60% on non-prod compute

DELIVERY:

- Week 2: Quick wins implemented (unused resources, shutdown scripts)
 - Typically saves 10–15% immediately
- Week 3–4: Right-sizing + Savings Plans analysis
 - Saves 15–20% with low risk
- Month 2–3: Architecture optimizations (caching, async, Spot/Graviton)
 - Saves another 10–15% with testing
- Total: 30–40% achievable in 3 months

ONGOING GOVERNANCE:

- Weekly cost review meeting (15 min, automated dashboard)
- Per-team cost dashboards (chargeback model)

- └─ Anomaly alerts: spike > 20% day-over-day → auto-alert
- └─ Quarterly commitment optimization review

4. Debugging & Incident Scenarios

Q8: "A microservice is returning intermittent 504 Gateway Timeout errors. It works fine 95% of the time. How do you diagnose it?"

Answer:

INTERMITTENT 504 – THE HARDEST KIND TO DEBUG:

WHY INTERMITTENT IS HARD:

504 means the upstream server didn't respond in time.

95% success rate = it's not fully broken. The 5% failures are likely:

- Load-dependent (only fails at certain concurrency)
- Time-dependent (specific time patterns)
- Data-dependent (certain request payloads trigger slow paths)
- Infrastructure-dependent (specific pod/node/AZ)

STEP 1: CORRELATE (15 min)

└─ CloudWatch / ALB access logs: filter for 504s

→ Is it one target (pod) or all targets?

→ Is it one URL path or all paths?

→ Is it one time window or random?

aws athena query on ALB logs:

SELECT target_ip, request_url, count(*) as errors

FROM alb_logs

WHERE elb_status_code = 504

AND time > now() - interval '24' hour

GROUP BY target_ip, request_url

ORDER BY errors DESC

└─ If ONE target: pod-specific problem (memory, thread pool, GC)

└─ If ONE path: slow query or expensive computation on that endpoint

└─ If ALL targets at specific times: database connection exhaustion or dependency failure

└─ If RANDOM: likely connection pool or DNS issue

STEP 2: TRACE THE SLOW REQUESTS (30 min)

└─ Distributed tracing (Jaeger / X-Ray):

→ Filter traces with duration > 30s (the timeout threshold)

→ Find: which span is the bottleneck?

- Common findings:
 - DB query span: 28 seconds (missing index, lock contention)
 - External API call: 30 seconds (downstream timeout)
 - No trace at all: request never reached the service (network)

— If DB:

```
kubectl exec -it <pod> -- psql -c "
  SELECT pid, now() - pg_stat_activity.query_start AS duration,
         query, state
  FROM pg_stat_activity
  WHERE state != 'idle'
  ORDER BY duration DESC LIMIT 10;"
```

- Long-running queries? Lock waits? Connection pool exhausted?

— If external dependency:

```
kubectl exec -it <pod> -- curl -w "\n
  time_namelookup: %{time_namelookup}\n
  time_connect: %{time_connect}\n
  time_appconnect: %{time_appconnect}\n
  time_total: %{time_total}\n" \
  -o /dev/null -s https://external-api.com/health
```

→ DNS resolution slow? TLS handshake slow? Response slow?

STEP 3: COMMON ROOT CAUSES FOR INTERMITTENT 504s

A. Connection pool exhaustion:

Max connections = 20, but under peak load 25 concurrent requests
 → 5 requests queue → some exceed ALB timeout → 504
 Fix: Increase pool size OR add circuit breaker (fail fast)

B. DNS resolution timeout:

CoreDNS overloaded → occasional 5-second DNS delay → triggers 504
 Fix: Set ndots: 2 in pod dnsConfig (reduces unnecessary DNS lookups)
 Validate: `kubectl logs -n kube-system -l k8s-app=kube-dns | grep SERVFAIL`

C. Java garbage collection pauses:

Full GC takes 2-5 seconds → pod unresponsive → ALB marks unhealthy
 Fix: `-XX:+UseG1GC -XX:MaxGCPauseMillis=200`
 Validate: GC logs, Prometheus `jvm_gc_pause_seconds_sum`

D. EKS node network throttling:

t3/t2 instances have burstable network (credit system)
 When credits exhausted → packet drops → 504
 Fix: Use m5/m6g (consistent network) instead of t3 for production

E. ALB idle timeout mismatch:

ALB idle timeout = 60s, but application keep-alive = 30s
 → ALB sends request on connection that app already closed → 504
 Fix: App keep-alive timeout > ALB idle timeout (set app to 90s)

5. Trade-off & Decision-Making

Q9: "Kafka vs. SQS+SNS vs. EventBridge — how do you choose for a specific client?"

Answer:

Factor	Kafka (MSK)	SQS+SNS	EventBridge
Throughput	Millions/sec	3,000/sec per queue (FIFO); unlimited (Standard)	400 req/sec (can increase)
Ordering	Per-partition	FIFO queues only	Per-rule (via SQS FIFO target)
Replay	Yes (configurable retention)	No (once consumed, gone)	Archive + replay (14 days)
Operational overhead	High (MSK) or Medium (Confluent Cloud)	Zero (fully managed)	Zero (fully managed)
Cost at scale	\$\$ (broker instances 24/7)	\$ (pay per message)	\$ (pay per event)
Schema registry	Confluent Schema Registry	None built-in	Schema Registry (Glue)
Consumer model	Pull (consumer groups, partitions)	Pull (long polling)	Push (rule-based routing)
Use case	High-throughput streaming, event sourcing, log aggregation	Task queues, point-to-point, simple pub/sub	Cross-service integration, routing, fan-out

My decision tree:

Need event replay or event sourcing? → Kafka

Need > 100K events/sec sustained? → Kafka

Need simple task queue with retries? → SQS

Need fan-out to multiple consumers? → SNS + SQS

Need cross-account/cross-service routing? → EventBridge

Need lowest operational overhead? → EventBridge or SQS

Starting a new project with < 10K events/sec? → EventBridge (simplest)

What I tell clients:

"Don't default to Kafka because it's powerful. Kafka is a distributed system that needs operational expertise. If your use case is 'Service A needs to tell Service B something happened,' SQS+SNS or EventBridge is 10x simpler. Reserve Kafka for genuine streaming use cases — click streams, log aggregation, event sourcing, CDC."

Q10: "Monorepo vs. polyrepo for 50+ microservices — what's your recommendation?"

Answer:

MONOREPO:

Pros:

- Single source of truth – atomic cross-service changes
- Shared tooling: one CI config, one linting setup, one dependency management
- Easy refactoring across services (IDE finds all references)
- Consistent versioning (all services in lockstep)
- Developer discoverability (one place to search)

Cons:

- CI complexity: must detect which services changed → build only those
- Repository size: slow clone for large codebases
- Blast radius: bad commit blocks all services
- Team ownership: harder to enforce per-team code ownership
- Requires investment in tooling (Bazel, Nx, Turborepo)

POLYREPO:

Pros:

- Team autonomy: each team owns their repo, their pipeline, their release
- Independent versioning and deployment
- Clear ownership (repo = team)
- Simple CI/CD per repo
- Smaller blast radius per commit

Cons:

- Cross-service changes require multi-repo PRs (painful)
- Dependency management: diamond dependency hell across repos
- Inconsistency: each team's CI/CD pipeline diverges
- Duplication: shared libraries, boilerplate, configs duplicated
- Discoverability: "where is this code?" requires searching 50 repos

MY RECOMMENDATION FOR XEBIA CLIENTS (50+ services):

Hybrid approach:

- Platform monorepo: all infrastructure code (Terraform, Helm charts, shared libraries, CI templates) in one repo
 - Platform team owns; provides "golden path" for all teams
- Domain polyrepos: grouped by bounded context
 - payments-domain/ (3 services: gateway, processor, reconciliation)
 - identity-domain/ (2 services: auth, user-management)
 - orders-domain/ (4 services: order, cart, pricing, fulfillment)
 - Each domain repo has 1-5 related services that change together
- Shared libraries: published as versioned packages (npm/maven/pip)
 - Consumed as dependencies, not source code imports
 - Versioned independently; consumers upgrade at their pace

6. Architecture Leadership at Scale

Q11: "How do you introduce a new technology (e.g., service mesh) into an organization that's resistant to change?"

Answer:

THE CHANGE MANAGEMENT PLAYBOOK:

Phase 1: BUILD THE CASE (don't sell the technology – sell the outcome)

- Identify the pain: "Our MTTR for cross-service issues is 3 hours because we can't trace requests across 30 services"
- Quantify the cost: "At 2 incidents/month × 3 hours × 5 engineers × \$100/hr
 - = \$36K/year in incident response alone"
- Propose the outcome: "Service mesh gives us automatic distributed tracing, mTLS security, and traffic management – reducing MTTR to 30 minutes"
- DON'T say: "We should use Istio because it's industry standard"

Phase 2: PROVE IT (PoC, not presentation)

- Install on ONE non-critical service in staging (2-3 days)
- Show: request tracing across 3 services in Jaeger dashboard
- Show: mutual TLS with zero code changes (security team loves this)
- Show: canary deployment shifting 10% traffic to new version
- Invite the skeptics to the demo (let them poke holes)
- Measure: latency overhead (typically < 2ms per hop)

Phase 3: EXPAND WITH CHAMPIONS

- Find 2–3 early adopter teams (not the resisters)
- Support them hands-on: pair with them during rollout
- Let them present results at engineering all-hands
- Their success becomes social proof for the organization
- Champions become the support network (not just you)

Phase 4: STANDARDIZE

- Create the "golden path": Helm chart with sensible defaults
- Document: runbook for common issues, FAQ, troubleshooting guide
- Automate: new services get mesh sidecar by default
- Train: 2-hour workshop for all engineering teams
- Measure adoption: % of services in mesh, MTTR improvement

Phase 5: HANDLE RESISTERS

- Listen to their concerns (usually: performance overhead, complexity)
- Address with data: "Latency overhead is 1.5ms; MTTR improvement is 85%"
- Offer escape hatch: "If your service can't tolerate 2ms overhead, we can exclude it – but you'll need to implement your own mTLS and tracing" (Usually the overhead of "doing it yourself" is higher than accepting mesh)
- Accept: not 100% adoption. 90% adoption with 10% exceptions is fine.

Q12: "How do you measure the health of an engineering organization's architecture?"

Answer:

ARCHITECTURE HEALTH SCORECARD (Quarterly):

DELIVERY METRICS (DORA):

- Deployment Frequency: daily (elite) → monthly (low)
- Lead Time for Changes: < 1 day (elite) → > 6 months (low)
- Change Failure Rate: < 5% (elite) → > 30% (low)
- MTTR: < 1 hour (elite) → > 1 week (low)

ARCHITECTURE QUALITY:

- Service coupling score: % of services with synchronous dependencies
Target: < 30% sync; > 70% async (event-driven)
- API contract coverage: % of services with published OpenAPI/AsyncAPI
Target: 100%
- Test coverage: meaningful (not just line coverage)
Unit: > 80%, Integration: > 60%, Contract: 100% of API boundaries
- Technical debt index: rated per service (1–5 scale)
Target: no service at 5 (critical debt)
- Dependency currency: % of dependencies within 2 major versions of latest
Target: > 90%

PLATFORM HEALTH:

- Developer onboarding time: hours to first commit in production

- | Target: < 4 hours
- Self-service adoption: % of infrastructure provisioned without tickets
- | Target: > 80%
- Incident toil: hours/week/engineer on operational work
- | Target: < 5 hours/week
- SLO compliance: % of services meeting their SLO targets
- | Target: > 95%
- Cost efficiency: cost per transaction (should decrease over time)

SECURITY HEALTH:

- Mean Time to Patch (critical CVE): target < 48 hours
- Secrets rotation compliance: target 100% automated
- Compliance audit findings: target < 5 per audit
- Security incidents: target 0 breaches

TEAM HEALTH:

- Developer satisfaction (quarterly survey)
- Retention rate of senior engineers
- Internal mobility (engineers growing within, not leaving)
- Knowledge sharing (ADRs written, tech talks delivered, PRs reviewed)

7. Xebia Culture & Values Fit

Q13: "What does 'craftsmanship' mean to you in the context of software architecture?"

Answer:

"Craftsmanship in architecture means three things to me:

1. Pride in the invisible work. Anyone can build a system that works.

Craftsmanship is in the things customers never see — error handling that's graceful, not crashy. Logging that tells you 'why' not just 'what'. APIs that are intuitive, not documented-because-they're-confusing. The test that catches the edge case before production does.

2. Leaving the codebase better than you found it. At Xebia, we're consultants — we work in clients' codebases and leave. Craftsmanship means leaving not just working code, but teaching patterns, documenting decisions, and building the client's team capability. A craftsman consultant makes themselves unnecessary.

3. Knowing when NOT to be clever. The most elegant solution is often the simplest one. Craftsmanship is choosing boring technology for the right reasons, writing code that a junior engineer can understand, and resisting the urge to over-engineer for problems that don't exist yet.

In practical terms: I review my own PRs 24 hours after writing them. If I can't understand my own code instantly, it's not crafted well enough."

Q14: "How do you contribute to the broader engineering community? (Open source, talks, writing, mentoring)"

Answer:

"I believe a Principal Architect's influence should extend beyond the current engagement:

Knowledge sharing:

- I write internal architecture case studies after every major engagement — anonymized, focused on patterns and anti-patterns. These become training material for Xebia's other architects.
- I maintain a 'tech radar' for my practice area — updated quarterly, shared with the team.

Mentoring:

- I actively mentor 2-3 senior engineers who aspire to architect roles. Weekly 1:1s focused on 'how to think architecturally' — not just technology.
- I run architecture katas — 90-minute exercises where teams design a system under constraints. Great for building architectural muscle.

Community:

- I contribute to open-source projects I use in client work — Terraform modules, Helm charts, Kubernetes operators. Even documentation improvements count.
- I present at internal tech talks and aim for 1-2 external meetup/conference talks per year.

At Xebia specifically, I'd:

- Contribute to Xebia's public technical blog
- Build reusable architecture accelerators (Terraform modules, reference architectures) for the consulting practice
- Mentor junior consultants on client engagement skills, not just technology"

8. Behavioral / STAR Stories

Story 1: "Tell me about a time you inherited a failing project and turned it around."

Situation: Joined a cloud migration engagement at month 4 (of 6). The previous architect had designed a complex microservices architecture with 35 services for a client that had 5 developers. Team morale was low. Client trust was broken. 2 of 6 months wasted on boilerplate.

Task: Deliver a working cloud platform in the remaining 2 months. Rebuild client trust.

Action:

1. **First 3 days: listened.** Met every developer and the CTO individually. Asked: "What's the most frustrating thing about this project?"
2. **Diagnosis:** Too many services for too few developers. Classic over-engineering. The team was spending 80% of time on cross-service plumbing and 20% on business logic.
3. **Reset:** Consolidated 35 planned services into 5 domain services (modular monoliths per domain). Same team, same timeline, but now each developer owned 1 service end-to-end.
4. **Architecture decision:** Documented the change in an ADR — "Why we chose domain-aligned services over fine-grained microservices" — co-authored with the client's lead developer.
5. **Quick win:** Got the first service deployed to EKS in 1 week. The client's CTO saw their code running in production on AWS for the first time. Visible progress rebuilt trust instantly.

6. **Execution:** Weekly demos to the CTO. No surprises. Transparent about risks.

Result: Delivered 4 of 5 domain services in 2 months. Client extended the engagement by 4 months for the remaining service + production hardening. Team velocity tripled. Client NPS went from -40 to +60.

Learning: Architecture must match organizational capacity. 35 services for 5 developers is an architecture failure, not a development failure. Conway's Law is real.

Story 2: "Tell me about a time you had to say 'no' to a client."

Situation: A client CTO asked us to skip load testing because "we need to launch by March 31st for the board meeting." The system was a real-time bidding platform — latency-critical, revenue-generating.

Task: Push back without damaging the relationship. The CTO signs our invoices.

Action:

1. **Didn't say 'no' directly.** Instead: "I completely understand the board deadline. Let me propose a way to do both — launch on time AND validate performance."
2. **Proposed:** Reduced load test scope — instead of full regression, we'd test only the 3 critical paths (bid submission, auction close, payment). 2 days instead of 2 weeks.
3. **Showed risk:** "If we skip testing and the system fails at 10x traffic on launch day, the board meeting conversation will be very different."
4. **Compromise:** Launch on March 31 with a feature flag — 10% of traffic to new system, 90% to old. Full load test runs March 28-30 in parallel. If the test passes, we open to 100% on April 2.

Result: Load test on March 29 revealed a connection pool bottleneck that would have caused outages at 5x traffic. Fixed in 4 hours. Launched at 10% on March 31, 100% on April 3. Zero incidents. CTO thanked us publicly at their all-hands.

Learning: "No" is rarely the right answer in consulting. "Yes, and here's a safer way to do it" preserves the relationship AND the technical integrity.

9. VP/CTO Rapid-Fire Questions

Question	Strong Answer
Why Xebia?	"Xebia's culture of craftsmanship aligns with how I work. I want to solve different architecture problems across industries, not the same problem at one company. And Xebia's investment in engineering excellence — not just delivery — is rare in consulting."
What's the biggest architecture mistake you see companies make?	"Adopting microservices before they have the organizational maturity to operate them. They get a distributed monolith — all the complexity, none of the benefits."
How do you handle working with a client architect who has more tenure but less skill?	"I respect their context. They know things about the system and organization that I never will. I bring outside patterns; they bring inside knowledge. We're complementary, not competitive."
What's the most underrated architecture pattern?	"The modular monolith. It gives you 80% of microservices benefits (modularity, team ownership, clear boundaries) with 20% of the complexity. Most systems should start here."
Rate yourself 1-10 on these: AWS, Kubernetes, Architecture, Leadership.	"AWS: 8 (deep hands-on, not every service). Kubernetes: 8.5 (production at scale, CKA-level). Architecture: 9 (this is my craft). Leadership: 8 (I'm still learning to let go and delegate more)."
What's a technology you're excited about right now?	"eBPF-based observability (Cilium, Tetragon). It gives you kernel-level visibility without the overhead of sidecars. This will change how we do security and networking in Kubernetes."
How do you keep 50+ teams aligned architecturally?	"Three layers: automated guardrails (OPA, Kyverno), architecture decision records (ADRs), and a tech radar. Teams have freedom within the guardrails. I'm an enabler, not a gatekeeper."

Question	Strong Answer
What would you do in your first 90 days at Xebia?	"Month 1: Listen and learn — shadow 2-3 active engagements, understand Xebia's delivery model. Month 2: Identify one repeatable accelerator I can build (reference architecture, Terraform module library). Month 3: Lead or co-lead a new engagement using what I've built."
If a client's system went down during your engagement, what's the first thing you do?	"Join the war room. Help debug, not direct. Take notes for the post-mortem. After it's resolved, help them build the monitoring that would have prevented it."
Biggest gap in your experience?	"I've done limited work with mainframe modernization — systems on COBOL/AS400. If Xebia has BFSI clients with mainframe estates, I'd need to ramp up on that modernization pattern."

10. Questions YOU Should Ask

About Xebia's Architecture Practice:

1. *"What does the architecture practice look like at Xebia today — how many Principal Architects, and how do they collaborate across engagements?"*
2. *"What's the typical engagement model — do architects stay on one client for 6-12 months, or rotate across multiple clients?"*
3. *"How does Xebia handle the 'consultant dependency' problem — ensuring clients are self-sufficient after we leave?"*

About the Role: 4. *"What does success look like for a Principal Architect at Xebia at the 6-month and 12-month mark?"*

5. *"Is there an expectation around pre-sales involvement — scoping, proposals, client pitches?"*
6. *"How much time is spent on delivery vs. internal practice building (thought leadership, accelerators, mentoring)?"*

About Client Work: 7. *"What industries are the current and target clients — BFSI, healthcare, retail, manufacturing?"*

8. *"What's the most complex architecture engagement Xebia has delivered in the last year, and what made it complex?"*

About Growth: 9. *"What does the growth path look like beyond Principal Architect — Distinguished Engineer? Practice Head? Engineering VP?"*

10. *"How does Xebia support continued learning — conference budgets, certification support, 20% time for OSS?"*

Final Interview Prep Checklist

48 hours before:

- ☐ Re-read L1 doc — refresh all 70 Q&As (skim, not deep-read)
- ☐ Practice 2 design sessions out loud (whiteboard or paper, 20 min each)
- ☐ Practice 2 STAR stories out loud (3 min each, timed)
- ☐ Review Xebia's website — recent blog posts, case studies, tech radar
- ☐ LinkedIn: research the interviewer — their background, interests, recent posts

Day of:

- ☐ Bring a notebook — take notes when the interviewer explains context
 - ☐ Mental label: 5 stories ready (Design, Debug, Client Conflict, Failure, Leadership)
 - ☐ When whiteboarding: start with boxes and arrows, not details. Ask before drawing.
 - ☐ Close strong: "I'm excited about bringing my architecture and consulting experience to Xebia. The combination of engineering craftsmanship and client diversity is exactly where I want to be."
-

The Difference Between L1 and L2 at Xebia

L1 (Technical Screen)	L2 (Final / Director)
"Explain CQRS"	"When would you NOT use CQRS?"
"How does Istio mTLS work?"	"A client says 'service mesh is too complex.' How do you respond?"
"Design a CI/CD pipeline"	"A team bypassed the pipeline to ship faster. What do you do?"
"What is event-driven architecture?"	"Kafka vs EventBridge for THIS specific client?"
Right answers	Right thinking
Technical depth	Consulting + architecture + leadership

*Prepared for: Pushparaj Naik | Role: Principal Architect — Xebia | Round: Final F2F
Complements: Xebia_Principal_Architect_Interview_Prep.md (L1 — 70 Q&As)
Prepared: June 2026*